

Deep Learning

Assignment #1

Deep Learning Basics

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 15 points*
- *Timeline: Assignment # 1 available in Week 3 (16th Sept), Code submission deadline till Week 5 (30th Sept), offline defense deadline till Week 6 (7th Oct)*

While completing this assignment, you will develop and strengthen the following skills:

- *Approaching a real-world machine learning problem with given data and a target metric.*
- *Conducting effective Exploratory Data Analysis (EDA).*
- *Designing a reliable validation strategy and monitoring it throughout the entire development process.*
- *Building a basic deep learning model and comparing it against traditional machine learning alternatives.*
- *Performing health checks on the optimization process.*
- *Implementing custom PyTorch layers and optimizers tailored to your specific problem.*
- *Creating a comprehensive performance report that highlights both final results and important optimization insights.*

Task

Task Selection:

- *Choose any task from Appendix 1. It is important to coordinate topics between teams so that there are no duplicates.*
- *Alternatively, you may propose your own task based on tabular data, **but it must be approved in advance by one of the lecturers.***

EDA and Metric Analysis:

- *Perform Exploratory Data Analysis (EDA).*
- *Analyze the proposed target metric: discuss its strengths, weaknesses, and potential edge cases.*

- Suggest complementary metrics to capture other aspects of performance.
- Propose an alternative target metric, with justification.

Validation:

- Design a validation strategy based on common sense and expected correlation with Private/Public leaderboard scores.
- Explain your motivation for the chosen strategy.
- Discuss possible reasons for low or high correlation with leaderboard results.
- Perform adversarial validation. If the competition is code-only, simulate a test set using the training data.

Machine Learning Baseline:

- Build a simple baseline model using tree-based algorithms (e.g., Random Forests, Gradient Boosting or something from sklearn). Perform dedicated feature engineering before training.
- Collect all relevant results:
 - metrics
 - feature importance,
 - parameter tuning logs, etc.
- Submit predictions to obtain Private/Public leaderboard metrics, and compare them with local validation results.
- Conclude whether the model is adequate.

Deep Learning Model:

- Build a deep learning model for the task. Do not limit yourself to a simple multilayer perceptron. You may include multiple input/output branches, auxiliary losses, [embeddings](#) (for categorical features), or other architectural ideas. Perform dedicated feature engineering and preprocessing.
- Implement at least:
 - One custom layer using [nn.Module](#) and `nn.Parameter` (can be original or a reimplementation of an existing layer),
 - One optimizer using [torch.optim.Optimizer](#) (can be original, modified, or reimplemented).
- Consider using Batch/Layer Normalizations, Dropouts, Weight Decays or any other regularizations and learning rate schedulers in your training process.
- Collect all relevant results:
 - loss/metric curves,
 - gradient flow visualizations,
 - parameter tuning logs, etc.
- Submit predictions and compare Private/Public leaderboard metrics with local validation results.
- Conclude whether the model is adequate.

Ensembling:

- Combine models from the two previous steps into an ensemble.
- Build a two-stage pipeline and ensure reliable validation for it.
- Submit predictions to obtain Private/Public leaderboard metrics, and compare them with local validation results.

Final Report

- Prepare a comprehensive report including:
 - data insights,

- feature engineering,
- tried models,
- chosen hyperparameters,
- metrics and optimization curves,
- model insights and conclusions.
- Reports may be prepared in LaTeX or Markdown ([README.md](#)).

NOTES

- For adversarial validation, Linear or Logistic Regression can be used.
- Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a “lucky shot.” Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself, and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	Points
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	8
Practical question about the submitted code during defense	2
Theoretical part during defense (2 questions + 1 exercise)	5
Criterion	Points

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student’s individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 2 and 1 Exercise from the Appendix 3, but will not necessarily be restricted to them.

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- *Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.*
- *Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.*
- *Language assistance tools: Grammarly, DeepL, Google Translate, etc.*
- *Local LLMs or self-hosted AI models.*

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- *Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.*
- *Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.*
- *Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.*
- *Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.*
- *Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.*
- *Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.*

Conditions for Use

Full Transparency:

- *Any use of AI tools must be disclosed in your submission.*
- *Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.*
- *If using online tools, include chat links (if available) or full transcripts in an appendix of your report.*

Partial Contribution Only:

- *AI tools should support your work – they must not produce the complete assignment, lab, or report.*
- *You must be able to explain and defend any AI-generated content during oral defense or Q&A*

Examples

Acceptable Uses:

- *"Correct grammar and style in this paragraph of my report."*
- *"Write a function to load images from a directory and resize them."*
- *"Suggest 3 ideas for improving model generalization."*

Unacceptable Uses:

- *"Write the entire peer review for this paper."*
- *"Generate the complete solution code for this assignment task."*

Appendix 1 (Recommended Tasks)

- <https://www.kaggle.com/competitions/home-credit-credit-risk-model-stability/overview>
- <https://www.kaggle.com/competitions/godaddy-microbusiness-density-forecasting/overview>
- <https://www.kaggle.com/competitions/amex-default-prediction/overview>
- <https://www.kaggle.com/competitions/ventilator-pressure-prediction/overview>
- <https://www.kaggle.com/competitions/lish-moa/overview>
- <https://www.kaggle.com/competitions/ashrae-energy-prediction>
- <https://www.kaggle.com/competitions/ieee-fraud-detection>
- <https://www.kaggle.com/competitions/home-credit-default-risk>
- <https://www.kaggle.com/competitions/champs-scalar-coupling>

Appendix 2 (Theoretical Questions)

- What is the main difference between Supervised and Unsupervised tasks ?
- When is empirical risk Q_{emp} equal to theoretical risk of the hypothesis Q ?
- A model has high accuracy (e.g. 95%) but low precision (e.g. 10%) and recall (e.g. 10%) on the positive class in a binary classification problem. What could be happening and should we describe this kind of performance as "rather good, because the accuracy is high" or "rather bad, because the precision/recall/f1 is low"?
- Describe cases (or provide examples) in which in a multiclass classification problem the values of F1 macro will be greater than F1 micro and vice versa? What do you think happens more often in practice and why?
- Imagine that you are in production developing a similar system for Twitter for identifying real disasters, think about whether the split proposed by the organizers (random, possibly with stratification by target) is optimal for a production model validation (assume that you have all the possible necessary data here) and if you think it could be not optimal - describe your ideas?
- In which case gradient descent will guide to global minimum ?
- A fully connected (linear) layer has NNN inputs and MMM outputs. How many trainable parameters does it have?
- Why do we include a bias term in a linear layer? In what scenarios might the bias be unnecessary?
- Why are activation functions needed in neural networks? Provide examples and briefly explain their roles.
- What happens to the optimization process at non-smooth points (e.g., with ReLU or absolute-value losses)?
- Explain how backpropagation works. Explain forward-mode automatic differentiation. In which situations is each preferable?
- Define static vs. dynamic computational graphs. Discuss the pros and cons of each approach.
- What can go wrong if ReLU is used as the final activation of a neural network?
- Define the hypothesis space. Why do we treat it as a space rather than a single function?
- Explain the difference between a sampled dataset and the underlying data distribution. How are they related?
- Compare PyTorch with NumPy, and PyTorch with TensorFlow: key similarities and differences.
- Give the definition of a convex function and list its key properties relevant to optimization.
- Prove that the negative gradient direction is a descent direction for a differentiable function.
- Prove that for convex functions, any local minimum is also a global minimum.
- Show that the Adam update defines a descent direction under appropriate assumptions (e.g., step size, moments, and smoothing).

- Explain why adaptive optimizers (Adam, RAdam, RMSProp) may perform worse than SGD on “simpler” or more nearly convex problems.
- Why is it important to track gradients during optimization? What insights and diagnostics does this provide?

Appendix 3 (Practical Exercises)

1. **Two-layer ReLU MLP:** Let $x = \begin{bmatrix} 1 \\ -2 \\ 0.5 \end{bmatrix}$, $W_1 = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \end{bmatrix}$, $b_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $z_1 = W_1x + b_1$, $a_1 = \text{ReLU}(z_1)$ with $\text{ReLU}(t) = \max(0, t)$. Second layer: $W_2 = \begin{bmatrix} 2 & -1 \end{bmatrix}$, $b_2 = 1$, output $\hat{y} = z_2 = W_2a_1 + b_2$. Target $y = 3$, loss $L = \frac{1}{2}(\hat{y} - y)^2$.

(a) **Forward:** Compute z_1 , a_1 , z_2 , $\hat{y}$.

(b) **Backprop (last layer only):** Compute $\frac{\partial L}{\partial W_2}$

2. **BatchNorm** Mini-batch scalars $x = \{1, 3, 5, 7\}$, $\gamma = 2$, $\beta = -1$, ε small.

$$\mu_B = \frac{1}{m} \sum_i x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_i (x_i - \mu_B)^2, \quad \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta.$$

Given upstream grads $\frac{\partial L}{\partial y} = \{1, 0, -1, 0\}$, compute

$$\frac{\partial L}{\partial \gamma} = \sum_i \frac{\partial L}{\partial y_i} \hat{x}_i, \quad \frac{\partial L}{\partial \beta} = \sum_i \frac{\partial L}{\partial y_i}.$$

(Leave any $\sqrt{\cdot}$ unevaluated.)

3. **Inverted dropout moment check** Keep-prob $q = \frac{1}{2}$, mask $m_j \sim \text{Bernoulli}(q)$ (independent), training output $a'_j = \frac{1}{q} m_j a_j$. For $a = [2, -1, 0, 3]$, show for each j :

$$\mathbb{E}[a'_j] = a_j, \quad \text{Var}(a'_j) = \frac{1-q}{q} a_j^2.$$

Evaluate numerically (with $q = \frac{1}{2}$ this is just a_j and a_j^2).

4. **L_2 penalty vs. decoupled weight decay** Let $w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$, gradient $g = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, step $\eta = \frac{1}{10}$, $\lambda = \frac{1}{10}$.

(a) L_2 penalty (added to loss): $w \leftarrow w - \eta(g + \lambda w)$.

(b) Decoupled weight decay: $w \leftarrow (1 - \eta\lambda)w - \eta g$.

Compute both updates exactly as fractions and compare.

5. **Cosine annealing with warm restarts** For a cycle of length T ,

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T)).$$

Let $\eta_{\max} = 0.1$, $\eta_{\min} = 0$, $T = 4$ in the first cycle. Compute η_t at $t = 0, 1, 2, 3, 4$. (*Hints:* $\cos 0 = 1$, $\cos \frac{\pi}{2} = 0$, $\cos \pi = -1$.)